



DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF ENGINEERING

ASSIGNMENT-1

Subject Name – System Design

Subject Code – 23CST- 314

Submitted To:

Er. Alok Kumar

(E17196)

Submitted By:

Name: Rakhi Channa

UID: 23BCS10431

Section: KRG_2-B

ASSIGNMENT:1

Q1. Explain the role of interfaces and Enums in software design with proper examples.

➔ Interfaces

Role in Software Design

- An interface defines a contract for classes without providing implementation.
- It specifies what a class should do, not how it does it.
- Interfaces are used for:
 - Achieving abstraction.
 - Supporting polymorphism.
 - Enabling loose coupling between components.
 - Allowing multiple inheritance of type (since a class can implement multiple interfaces).

Example

```
1 // Define an interface.
2 interface Vehicle {
3     void start();
4     void stop();
5 }
6
7 // Implement the interface in a class
8 class Car implements Vehicle {
9     public void start() {
10         System.out.println("Car is starting");
11     }
12     public void stop() {
13         System.out.println("Car has stopped");
14     }
15 }
16
17 // Implement another class
18 class Bike implements Vehicle {
19     public void start() {
20         System.out.println("Bike is starting");
21     }
22     public void stop() {
23         System.out.println("Bike has stopped");
24     }
25 }
26
27 // Usage
28 public class TestInterface {
29     public static void main(String[] args) {
30         Vehicle myCar = new Car();
31         Vehicle myBike = new Bike();
32         myCar.start();    // Car is starting
33         myBike.start();   // Bike is starting
34     }
35 }
```

➔ Enums

Role in Software Design

- Enum (enumeration) is a special data type that represents a fixed set of constants.
- Enums improve readability, type safety, and reduce errors compared to using integers or strings.
- Useful when a variable can only take specific predefined values.

Example

```
1 // Define an enum
2 enum Day {
3     MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
4 }
5
6 // Use enum in a class
7 class Schedule {
8     Day meetingDay;
9
10    Schedule(Day meetingDay) {
11        this.meetingDay = meetingDay,
12    }
13    void showMeeting() {
14        System.out.println("Meeting is on: " + meetingDay );
15    }
16 }
17
18 // Usage
19 public class TestEnum {
20     public static void main (String[] args) {
21         Schedule s = new Schedule(Day.MONDAY);
22         s.showMeeting(); // Meeting is on: MONDAY
23     }
24 }
```

Q2. Discuss how interfaces enable loose coupling with example.

Loose Coupling

Definition

- Loose coupling means that components of a system depend on abstractions, not concrete implementations.
- It makes the system flexible, maintainable, and easy to extend.

- Changes in one part of the system don't affect other parts if they interact through interfaces.

How Interfaces Enable Loose Coupling

1. An interface acts as a contract.
 - The client code knows what operations are available, not how they are implemented.
2. The client can interact with any class that implements the interface.
 - It doesn't care about the actual implementation.
3. New implementations can be added without changing existing client code.

Example: Payment System

Problem: We have an e-commerce app that can support multiple payment methods (Credit Card, PayPal, UPI).

Step 1: Define an Interface

```
interface Payment {  
    void pay(double amount);  
}
```

Step 2: Implement Multiple Payment Methods

```
class CreditCardPayment implements Payment {  
    public void pay(double amount) {  
        System.out.println("Paid " + amount + " using Credit Card");  
    }  
}
```

```
class PayPalPayment implements Payment {  
    public void pay(double amount) {  
        System.out.println("Paid " + amount + " using PayPal");  
    }  
}
```

```
class UPIPayment implements Payment {  
    public void pay(double amount) {  
        System.out.println("Paid " + amount + " using UPI");  
    }  
}
```

Step 3: Use Interface in Client Code

```
class ShoppingCart {
    private Payment paymentMethod;

    // Constructor injection
    public ShoppingCart(Payment paymentMethod) {
        this.paymentMethod = paymentMethod;
    }

    public void checkout(double amount) {
        paymentMethod.pay(amount); // Only depends on Payment interface
    }
}

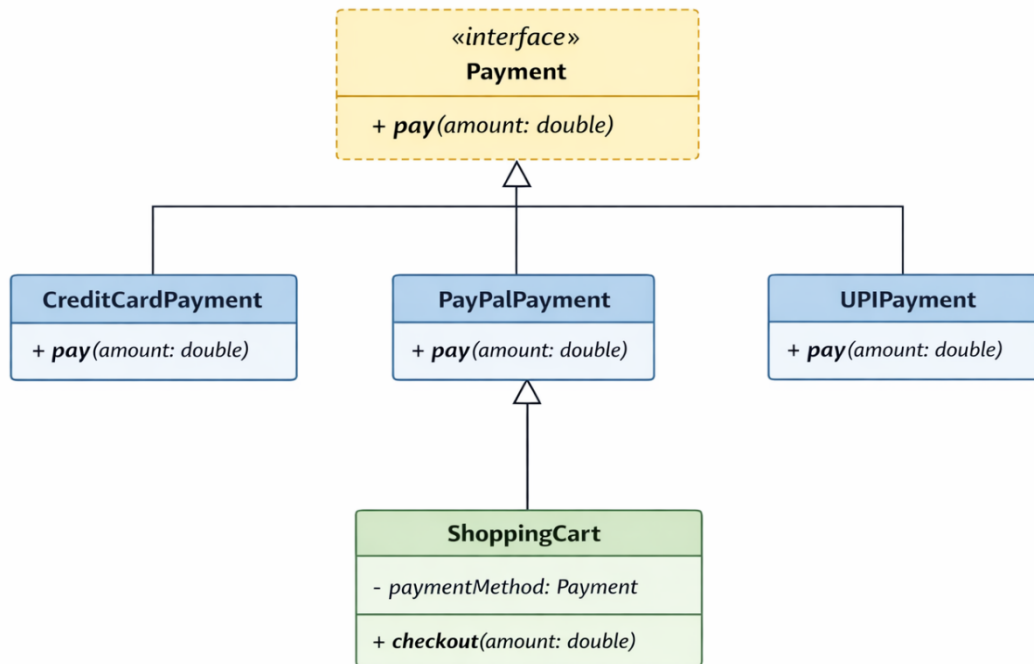
// Usage
public class TestPayment {
    public static void main(String[] args) {
        Payment credit = new CreditCardPayment();
        Payment paypal = new PayPalPayment();

        ShoppingCart cart1 = new ShoppingCart(credit);
        ShoppingCart cart2 = new ShoppingCart(paypal);

        cart1.checkout(500); // Paid 500 using Credit Card
        cart2.checkout(1200); // Paid 1200 using PayPal
    }
}
```

Why This is Loose Coupling

1. ShoppingCart depends only on Payment interface, not on concrete classes.
2. Adding new payment methods (e.g., BitcoinPayment) doesn't require changing ShoppingCart.
3. Implementation details of CreditCardPayment or PayPalPayment are hidden from ShoppingCart.



Q3. Design an HLD for a Payment Processing System, showing where interfaces would be used.

Functional Requirements (What system should do)

1. **User Authentication**
 - Verify user identity before payment.
 - Support login/session validation.
2. **Payment Processing**
 - Accept multiple payment modes (Card, UPI, Wallet, Net Banking).
 - Process payment requests.
3. **Transaction Management**
 - Generate transaction ID.
 - Track payment status (Pending, Success, Failed).
4. **Fraud Detection**
 - Check suspicious transactions.
 - Block risky payments.
5. **Refund Handling**

- Allow refund requests.
- Track refund status.
- 6. **Notification Service**
 - Send payment confirmation/failure alerts.
- 7. **Payment Gateway Integration**
 - Connect with external payment providers.

Non-Functional Requirements (How system should perform)

1. **Scalability**
 - Handle thousands/millions of transactions.
2. **Security**
 - Data encryption.
 - PCI-DSS compliance.
 - Secure authentication.
3. **Reliability**
 - Payment must not fail or lose data.
4. **Availability**
 - 24/7 uptime.
5. **Performance**
 - Fast payment response time.
6. **Maintainability**
 - Easy to add new payment methods using interfaces.
7. **Fault Tolerance**
 - System should recover from failures.

Where Interfaces Are Used in HLD

- IPaymentProcessor → Common interface for all payment methods
- IGateway → Interface for external payment providers
- IFraudCheck → Interface for fraud detection modules
- IAuth → Interface for authentication services

These help in **loose coupling** and easy extension.

Designed the required HLD in attached .drawio file.